

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ
УНИВЕРСИТЕТ «МИФИ»
(НИЯУ МИФИ)

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Методы оптимизации» на тему
«ВЫПУКЛЫЕ МНОЖЕСТВА И ИХ СВОЙСТВА»

Выполнил: студент группы Б24-215

(подпись)

Гаганидзе Г. Г.

(Фамилия И.О.)

Проверил:

(оценка)

(подпись)

Климанов С. Г.

(Фамилия И.О.)

Москва 2026 г.

Оглавление

1.	Введение	2
1.1.	Цель работы	2
2.	Теоретическая часть	2
2.1.	Выпуклое множество	2
2.2.	Выпуклые комбинации и выпуклая оболочка	2
2.3.	Пересечение выпуклых множеств	3
2.4.	Проверка выпуклости многоугольника	3
2.5.	Ответы на теоретические контрольные вопросы	4
3.	Методы	4
3.1.	Программные средства	4
3.2.	Задача 1: выпуклая оболочка	5
3.3.	Задача 2: пересечение кругов	6
3.4.	Задача 3: критерий по ориентации	7
4.	Результаты	8
4.1.	Задача 1	8
4.2.	Задача 2	9
4.3.	Задача 3	10
5.	Заключение	11

1. Введение

1.1. Цель работы

Целью лабораторной работы является экспериментальное исследование выпуклых множеств и их свойств с использованием языка Python и библиотек NumPy, SciPy и Matplotlib.

В работе решаются следующие задачи:

- построение выпуклой оболочки случайного набора точек на плоскости;
- визуальная проверка выпуклости пересечения двух кругов;
- программная проверка выпуклости многоугольника

Кроме реализации алгоритмов, необходимо сопоставить вычислительные результаты с теоретическими свойствами выпуклых множеств и ответить на контрольные вопросы из технического задания.

2. Теоретическая часть

2.1. Выпуклое множество

Определение 2.1 (Выпуклое множество) Множество S в вещественном векторном пространстве называется выпуклым, если выполняется:

$$\lambda x + (1 - \lambda)y \in S, \quad \forall x, y \in S, \quad \forall \lambda \in [0, 1].$$

Иными словами, вместе с любыми двумя своими точками множество содержит весь отрезок, соединяющий эти точки.

Примерами выпуклых множеств на плоскости являются прямая, полуплоскость, круг, эллипс, треугольник и выпуклый многоугольник. Невыпуклыми являются кольцо, два непересекающихся круга, область с выемкой и звёздный многоугольник: в каждом случае можно подобрать две точки множества, отрезок между которыми частично выходит за его границу.

2.2. Выпуклые комбинации и выпуклая оболочка

Определение 2.2 (Выпуклая комбинация) Выпуклой линейной комбинацией точек $x_1, \dots, x_k$ называется точка

$$x = \sum_{i=1}^k \lambda_i x_i, \quad \sum_{i=1}^k \lambda_i = 1, \quad \lambda_i \geq 0, \quad i = \overline{1, k}.$$

Для двух точек эта формула задаёт отрезок: $x = \lambda x_1 + (1 - \lambda)x_2$. Для трёх точек она задаёт точки треугольника с вершинами x_1, x_2, x_3 , включая его границу.

Определение 2.3 (Выпуклая оболочка) *Выпуклая оболочка множества точек — это наименьшее выпуклое множество, содержащее исходное множество.*

Эквивалентно, она состоит из всех выпуклых комбинаций исходных точек. Поэтому в первой задаче выпуклая оболочка случайных точек представляется выпуклым многоугольником, вершины которого являются крайними точками исходного набора.

2.3. Пересечение выпуклых множеств

Теорема 2.1 (О пересечении выпуклых множеств) *Пусть $\{S_\alpha\}_{\alpha \in I}$ — семейство выпуклых множеств. Тогда их пересечение*

$$S = \bigcap_{\alpha \in I} S_\alpha$$

является выпуклым множеством.

Доказательство 2.1 *Пусть $x, y \in S$ и $\lambda \in [0, 1]$. По определению пересечения $x, y \in S_\alpha$ для любого $\alpha \in I$.*

Так как каждое множество S_α выпукло, то

$$\lambda x + (1 - \lambda)y \in S_\alpha \quad \text{для любого } \alpha \in I.$$

Следовательно,

$$\lambda x + (1 - \lambda)y \in \bigcap_{\alpha \in I} S_\alpha = S.$$

Таким образом, S является выпуклым множеством.

Это свойство позволяет одновременно учитывать несколько выпуклых ограничений в задачах оптимизации: область допустимых решений сохраняет удобную геометрическую структуру.

2.4. Проверка выпуклости многоугольника

Для последовательных вершин A, B, C вычисляется ориентированное двумерное векторное произведение

$$(B - A) \times (C - B) = (B_x - A_x)(C_y - B_y) - (B_y - A_y)(C_x - B_x).$$

Его знак показывает направление поворота при переходе от ребра AB к ребру BC . У правильно заданного выпуклого многоугольника все ненулевые повороты имеют один знак: все повороты происходят против или по часовой стрелке. Смена знака означает наличие внутреннего угла, направленного наружу, и, следовательно, невыпуклость.

Нулевое произведение соответствует коллинеарным соседним рёбрам и не меняет направление обхода. В реализованной функции такие случаи пропускаются, поэтому многоугольники с промежуточными точками на одной стороне могут считаться выпуклыми.

2.5. Ответы на теоретические контрольные вопросы

1. Определение и примеры. Выпуклое множество содержит отрезок между любой парой своих точек. Выпуклыми примерами служат круг, квадрат и полуплоскость; невыпуклыми — кольцо, звёздная область и многоугольник с выемкой.

2. Пересечение. Согласно теореме 2.1, пересечение выпуклых множеств выпукло, потому что для любой пары точек пересечения отрезок между ними одновременно лежит в каждом из исходных множеств. На практике это важно при формировании области допустимых решений системы ограничений.

3. Геометрический смысл комбинации. Коэффициенты выпуклой комбинации являются неотрицательными весами, сумма которых равна единице. Поэтому результат является взвешенным средним исходных точек и находится в их выпуклой оболочке. Все точки выпуклой оболочки можно получить таким способом.

4. Методы проверки. Используются проверка углов и знаков векторных произведений, ориентационный тест, проверка пересечений рёбер и построение выпуклой оболочки с последующим сравнением. В работе применён линейный проход по последовательным тройкам вершин с фиксацией первого ненулевого знака.

5. Практическая значимость. Выпуклость упрощает оптимизацию: у выпуклой задачи локальный минимум является глобальным, а область допустимых решений не содержит раздельных локальных подобластей. Выпуклые множества используются в линейном и квадратичном программировании, методах опорных векторов, регуляризации и анализе ограничений машинного обучения.

3. Методы

3.1. Программные средства

Для вычислений использовались Python, NumPy, SciPy и Matplotlib. SciPy применялся для построения выпуклой оболочки алгоритмом ConvexHull, NumPy — для генерации точек и векторизованной проверки принадлежности кругам, Matplotlib — для построения и сохранения графиков.

3.2. Задача 1: выпуклая оболочка

Генерируются 20 точек с координатами из диапазона $[-5, 5]$. Функция `ConvexHull` возвращает индексы вершин оболочки в порядке обхода. Внутренние точки обозначены синим прозрачным цветом, а вершины оболочки — красным. Сама оболочка закрашена полупрозрачным зеленым цветом.

```
1 #... code/lab02-1.py
2 import numpy as np
3 from matplotlib import pyplot as plt
4 from scipy.spatial import ConvexHull
5
6 #...
7 points = np.random.rand((N, 2)) * 10.0 - 5.0
8
9 hull = ConvexHull(points)
10 hull_points = hull.points
11
12 vertex_indices = hull.vertices
13 hull_vertices = points[vertex_indices]
14
15 mask = np.zeros(len(points), dtype=bool)
16 mask[vertex_indices] = True
17
18 inner_points = points[~mask]
19
20 plt.figure(figsize=(8, 6))
21 plt.scatter(inner_points[:, 0], inner_points[:, 1], label="Random Points")
22 plt.scatter(hull_vertices[:, 0], hull_vertices[:, 1], label="Convex Hull
    ↪ Points")
23 plt.fill(points[vertex_indices][:, 0], points[vertex_indices][:, 1], alpha=0.3,
    ↪ label="Convex Hull")
24 plt.title("Convex Hull of Random Points")
25 #...
26 plt.show()
```

3.3. Задача 2: пересечение кругов

Рассматриваются множества

$$A = \{(x, y) : x^2 + y^2 \leq 3^2\},$$

$$B = \{(x, y) : (x - 2)^2 + (y - 1)^2 \leq 2.5^2\}.$$

На сетке $[-5, 5] \times [-5, 5]$ с шагом 0.01 для каждой точки вычисляются логические маски принадлежности A , B и $A \cap B$. Три непересекающиеся категории отображаются разными цветами.

```
1 #... code/lab02-2.py
2 import numpy as np
3 from matplotlib import pyplot as plt
4 from matplotlib.patches import Patch
5 from matplotlib.colors import ListedColormap
6
7
8 def in_A(x, y):
9     return x * x + y * y <= 9.0
10
11 def in_B(x, y):
12     return (x - 2.0) ** 2 + (y - 1.0) ** 2 <= 6.25
13
14 x_min, x_max = -5.0, 5.0
15 y_min, y_max = -5.0, 5.0
16 step = 0.01
17 Y, X = np.mgrid[y_min:y_max:step, x_min:x_max:step]
18
19 mask_A, mask_B = in_A(X, Y), in_B(X, Y)
20
21 img = np.zeros(X.shape, dtype=int)
22 img[mask_A & ~mask_B], img[mask_B & ~mask_A], img[mask_A & mask_B] = 1, 2, 3
23
24 #... Настройка цветов и легенды
25
26 plt.figure(figsize=(8, 8))
27 plt.imshow(img, extent=(x_min, x_max, y_min, y_max), origin="lower", cmap=cmap)
28 plt.title("Sets A, B and A ∩ B")
29 plt.grid()
30 plt.legend(handles=legend_elements, loc="upper right")
31 plt.show()
```

3.4. Задача 3: критерий по ориентации

Функция `is_convex` последовательно рассматривает все тройки соседних вершин, включая переход через последнюю вершину. Для каждой тройки вычисляется знак ориентированного произведения. Первый ненулевой знак сохраняется как эталонный; если встречается противоположный знак, функция немедленно возвращает `False`. В тестах использованы выпуклый многоугольник с коллинеарной точкой, невыпуклый многоугольник с выемкой и звёздный многоугольник.

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3
4
5 def is_convex(points):
6     n = len(points)
7
8     if n < 3:
9         return False
10
11     orientation = 0
12
13     for i in range(n):
14         edge_a = points[(i + 1) % n] - points[i]
15         edge_b = points[(i + 2) % n] - points[(i + 1) % n]
16
17         current_orientation = orientation_of_cross(edge_a, edge_b)
18
19         if current_orientation == 0:
20             continue
21
22         if orientation == 0:
23             orientation = current_orientation
24         elif current_orientation != orientation:
25             return False
26
27     return orientation != 0
28
29
30 def orientation_of_cross(vector_a, vector_b):
31     x1, y1 = vector_a
32     x2, y2 = vector_b
33     oriented_length = x1 * y2 - y1 * x2
34     return np.sign(oriented_length)
```

```

1 # Одна точка лежит на линии между двумя другими точками,
2 # но не нарушает выпуклость.
3 # Повторяющиеся точки не нарушают выпуклость
4 oriented_points_convex = np.array(
5     [[0, 0], [1, 0], [1, 1], [0.5, 1], [0, 1], [0, 0]]
6 )
7 oriented_points_non_convex = np.array(
8     [[0, 0], [1, 0], [0.5, 0.5], [1, 1], [0, 1], [0, 0]]
9 )
10 star_points = np.array(...)
11 print(is_convex(oriented_points_convex)) # True
12 print(is_convex(oriented_points_non_convex)) # False
13 print(is_convex(star_points)) # False

```

4. Результаты

4.1. Задача 1

Результат построения выпуклой оболочки случайных точек приведён на рисунке 1. Красные точки являются вершинами оболочки, а синие точки лежат внутри или на границе полученного многоугольника.

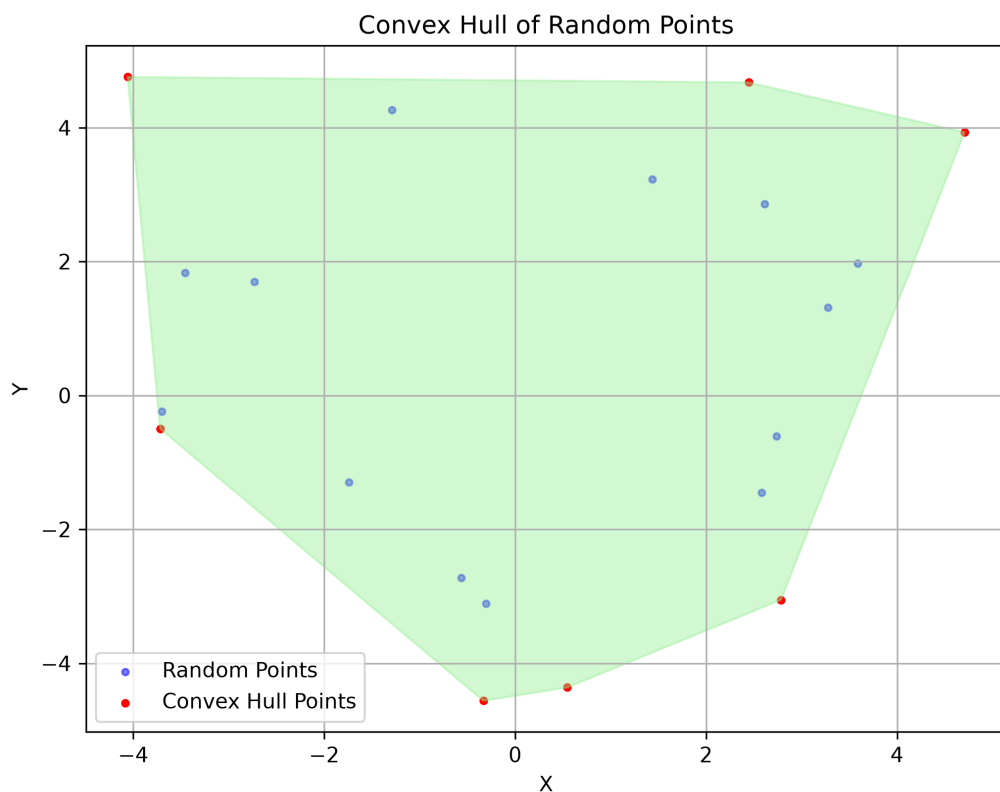


Рис. 1. Выпуклая оболочка 20 случайных точек

Полученная фигура является выпуклым многоугольником. Действительно, `ConvexHull` строит наименьший выпуклый многоугольник, содержащий все исходные точки. Поэтому для любых двух точек внутри закрашенной области соединяющий их отрезок не выходит за границу оболочки. Это непосредственно соответствует определению выпуклого множества.

4.2. Задача 2

На рисунке 2 синим цветом показана часть круга A , не принадлежащая B , красным — часть B , не принадлежащая A , фиолетовым — пересечение $A \cap B$.

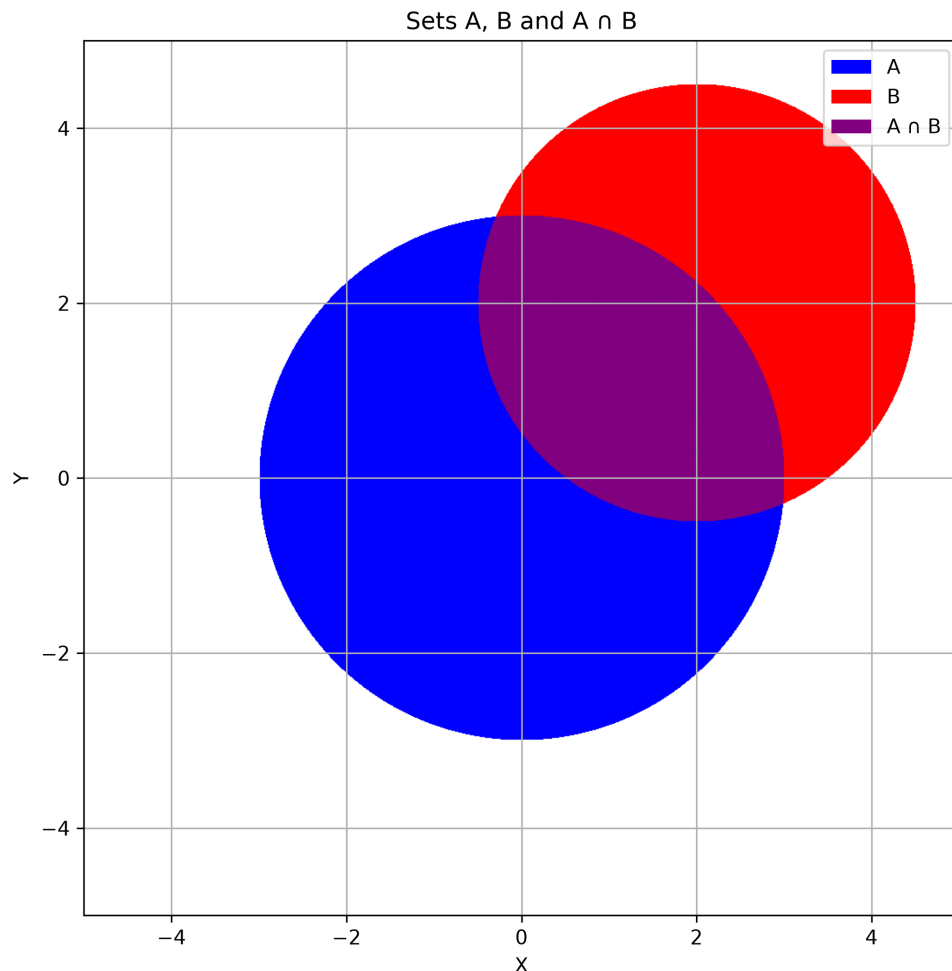


Рис. 2. Круги A , B и их пересечение

Оба круга выпуклы: для любых двух точек одного круга отрезок между ними также находится в этом круге. Их пересечение также выпукло по доказанному в теоретической части свойству. На графике фиолетовая область имеет форму линзовидного пересечения без выемок, что согласуется с теорией.

Следовательно, ответ на проверочный вопрос положительный: пересечение двух данных кругов является выпуклым множеством. Более того, пересечение любого числа выпуклых множеств всегда выпукло, хотя оно может быть пустым.

4.3. Задача 3

Результат печати программы:

True

False

False

Первый результат соответствует выпуклому многоугольнику. В нём присутствует коллинеарная промежуточная точка и повтор первой вершины в конце списка, однако нулевые повороты игнорируются, а направление остальных поворотов сохраняется. Второй результат показывает, что многоугольник с выемкой невыпуклый: знак поворота меняется. Третий результат подтверждает невыпуклость звезды, у которой последовательные рёбра также дают повороты разных направлений.

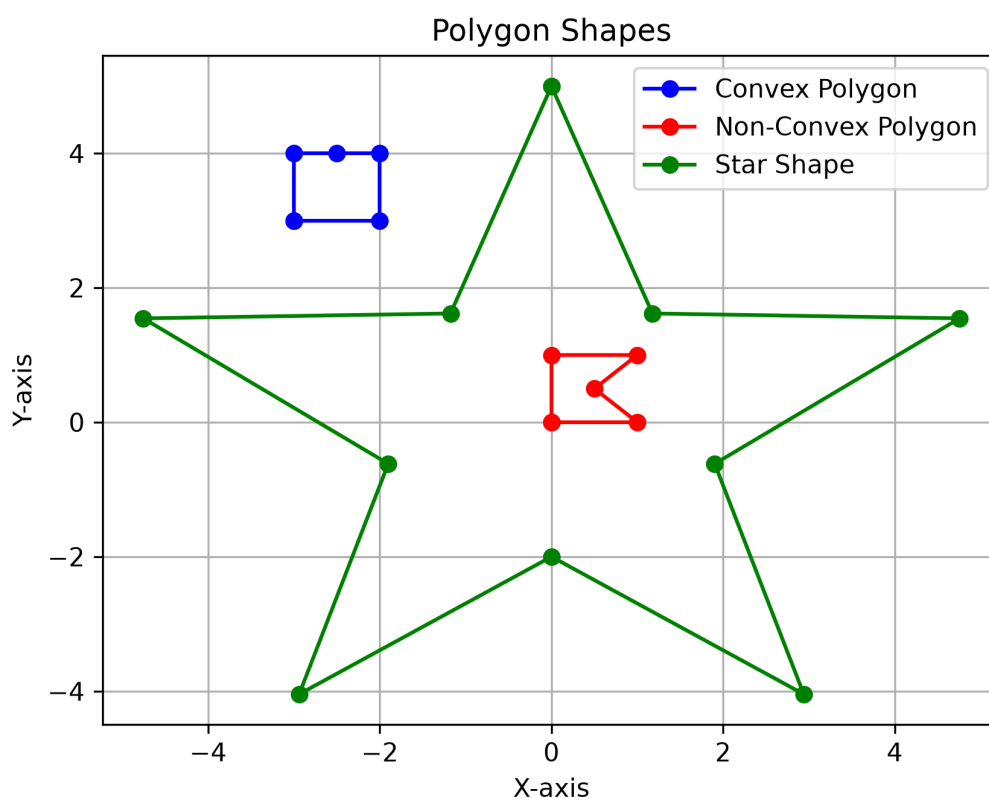


Рис. 3. Фигуры, использованные для проверки критерия выпуклости

Таким образом, реализация критерия правильно различает выпуклый многоугольник, многоугольник с выемкой и звёздную фигуру на выбранных тестах.

5. Заключение

В ходе лабораторной работы были изучены определение выпуклого множества, выпуклые линейные комбинации, выпуклая оболочка и свойство сохранения выпуклости при пересечении. Теоретические положения были проверены вычислительными экспериментами.

В первой задаче для 20 случайных точек была построена выпуклая оболочка. Полученный многоугольник содержит все исходные точки и является выпуклым по построению. Во второй задаче были заданы два выпуклых круга, а их принадлежность точкам плоской сетки визуализирована. Форма общей части и теорема о пересечении подтверждают, что пересечение выпуклых множеств выпукло. В третьей задаче реализована проверка знаков ориентированных произведений: тесты дали ожидаемые результаты True, False, False для выпуклой, невыпуклой и звёздной фигур соответственно.

Практический результат работы — набор воспроизводимых программ и графиков. Использование выпуклости позволяет надёжно описывать области допустимых решений и применять эффективные методы оптимизации: для выпуклых задач локальные свойства дают глобальные гарантии, а пересечение ограничений сохраняет выпуклую структуру.

Ограничение реализованного критерия состоит в том, что он рассчитан на упорядоченный список вершин простого многоугольника. Для произвольного неупорядоченного набора точек сначала следует построить выпуклую оболочку, а для самопересекающихся контуров требуется отдельная проверка корректности границы.

Литература

- [1] Сухарев А. Г., Тимохов А. В., Федоров В. В. *Курс методов оптимизации*. — Москва: Наука, 1986.